

Decision Support System
(DSS) Distribusi Bali
Graph



INSTIKI
INSTITUT BISNIS DAN
TEKNOLOGI INDONESIA

Oleh Trio PBZ:

I Gusti Ngurah Ketut Puja Bagus Permana (2501010021)

I Wayan Bayu Kusumadinatha (2501010359)

Zein Ahmad (2501010360)

PRODI INFORMATIKA

Institut Bisnis dan Teknologi Indonesia

TAHUN 2025/2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan proyek yang berjudul **“Decision Support System (DSS) Distribusi Bali Berbasis Graph”** dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas mata kuliah sekaligus sebagai implementasi penerapan struktur data graph dan algoritma Dijkstra pada sistem pendukung keputusan dalam menentukan rekomendasi tempat makan terbaik berdasarkan berbagai kriteria yang telah ditentukan.

Kami menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi materi maupun penyusunan. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang.

Kami berharap laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi dalam pengembangan sistem pendukung keputusan berbasis graph, khususnya dalam bidang rekomendasi tempat makan.

Akhir kata, kami mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini.

Bali, Mei 2026

Penyusun

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era modernisasi logistik dan distribusi barang, efisiensi rute pengiriman menjadi faktor kritis yang menentukan biaya operasional dan kepuasan pelanggan. Pulau Bali, sebagai salah satu destinasi wisata dan pusat ekonomi regional terbesar di Indonesia, memiliki jaringan distribusi yang kompleks yang mencakup berbagai wilayah dari pusat kota hingga kawasan terpencil seperti Nusa Penida.

Permasalahan optimasi rute merupakan salah satu contoh nyata penerapan teori graf dalam ilmu komputer. Sebuah jaringan distribusi dapat dimodelkan sebagai graf berbobot, di mana simpul (node) merepresentasikan titik distribusi dan sisi (edge) merepresentasikan jalur yang menghubungkan titik-titik tersebut dengan jarak tertentu. Dalam konteks ini, algoritma Dijkstra terbukti efektif untuk menemukan jalur terpendek antara dua titik dalam graf berbobot non-negatif.

Decision Support System (DSS) merupakan sistem informasi yang dirancang untuk mendukung pengambilan keputusan berbasis data. Dengan mengintegrasikan DSS ke dalam sistem manajemen distribusi, operator logistik dapat menentukan rute pengiriman yang optimal secara cepat dan akurat tanpa harus melakukan perhitungan manual yang memakan waktu.

Proyek ini membangun sebuah DSS berbasis graf menggunakan bahasa pemrograman Python dengan antarmuka Streamlit, yang memungkinkan pengguna untuk secara interaktif memilih titik asal dan tujuan pengiriman, kemudian sistem secara otomatis menghitung dan menampilkan rute terpendek beserta visualisasi grafisnya. Sistem dimodelkan berdasarkan jaringan distribusi di Pulau Bali, mencakup satu depot utama (Depot Denpasar), empat hub transit (Tabanan, Gianyar, Badung, Klungkung), dan delapan zona pengiriman akhir.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah:

1. Bagaimana merepresentasikan jaringan distribusi logistik di Pulau Bali ke dalam struktur data graf yang efisien?
2. Bagaimana mengimplementasikan algoritma Dijkstra untuk menentukan rute pengiriman terpendek antara dua titik distribusi dalam jaringan tersebut?
3. Bagaimana merancang antarmuka DSS yang interaktif dan informatif sehingga operator dapat dengan mudah memilih titik asal-tujuan dan memahami hasil rekomendasi rute?
4. Bagaimana memvisualisasikan graf jaringan distribusi beserta rute terpendek yang dihasilkan secara grafis?

1.3 Tujuan

Tujuan dari pengembangan sistem ini adalah:

- Membangun representasi jaringan distribusi Bali dalam bentuk graf berbobot menggunakan struktur adjacency list.
- Mengimplementasikan algoritma Dijkstra secara efisien dengan bantuan priority queue (min-heap) untuk mencari rute terpendek.
- Mengembangkan antarmuka web interaktif berbasis Streamlit yang memudahkan operator logistik dalam menggunakan sistem.
- Mengintegrasikan komponen visualisasi graf menggunakan library NetworkX dan Matplotlib agar pengguna dapat memahami topologi jaringan dan hasil rekomendasi secara visual.
- Menyediakan informasi detail langkah demi langkah (step-by-step) perjalanan rute beserta jarak setiap segmen.

1.4 Manfaat

Manfaat yang diharapkan dari sistem DSS ini adalah:

1.4.1 Manfaat Praktis

- Membantu operator logistik dalam menentukan rute pengiriman yang efisien, menghemat bahan bakar, dan meminimalkan biaya operasional.
- Mempercepat proses perencanaan rute yang sebelumnya dilakukan secara manual menjadi berbasis sistem yang otomatis dan akurat.
- Memberikan transparansi rute kepada pelanggan maupun manajemen melalui visualisasi grafis yang mudah dipahami.

1.4.2 Manfaat Akademis

- Membuktikan penerapan nyata struktur data graf dan algoritma Dijkstra dalam konteks permasalahan logistik dunia nyata.
- Menjadi referensi implementasi DSS berbasis algoritma graf menggunakan teknologi Python modern (Streamlit, NetworkX, Matplotlib).

Memberikan pemahaman mendalam tentang kompleksitas algoritma dan perbandingan pendekatan solusi pada masalah shortest-path

BAB II

DASAR TEORI

2.1 Struktur Data Graf

Graf (Graph) adalah struktur data non-linear yang terdiri dari himpunan simpul (vertex/node) dan himpunan sisi (edge/arc) yang menghubungkan pasangan simpul. Secara formal, sebuah graf G didefinisikan sebagai $G = (V, E)$, di mana V adalah himpunan simpul dan E adalah himpunan sisi.

2.1.1 Klasifikasi Graf

Berdasarkan sifat sisinya, graf diklasifikasikan menjadi beberapa jenis:

- Graf Tak Berarah (Undirected Graph): Sisi tidak memiliki arah, hubungan bersifat simetris. Jika terdapat sisi (u, v) , maka terdapat pula sisi (v, u) . Graf yang digunakan dalam sistem ini adalah undirected graph.
- Graf Berarah (Directed Graph / Digraph): Sisi memiliki arah, representasi hubungan satu arah. Contoh: jaringan lalu lintas satu arah.
- Graf Berbobot (Weighted Graph): Setiap sisi memiliki nilai bobot yang merepresentasikan cost perjalanan, misalnya jarak dalam kilometer, waktu tempuh, atau biaya bahan bakar.

2.1.2 Representasi Graf

Terdapat dua cara utama merepresentasikan graf dalam memori komputer:

- Adjacency Matrix: Matriks berukuran $|V| \times |V|$, di mana elemen $[i][j]$ berisi bobot sisi antara simpul i dan j , atau 0/infinity jika tidak ada sisi. Kompleksitas ruang: $O(V^2)$.
- Adjacency List: Setiap simpul menyimpan daftar tetangganya beserta bobot sisi. Lebih efisien untuk graf jarang (sparse graph). Kompleksitas ruang: $O(V + E)$. Inilah yang digunakan dalam proyek ini menggunakan Python dictionary.

Dalam proyek ini, adjacency list diimplementasikan menggunakan nested dictionary Python:

```
GRAPH = {
    "Depot Denpasar": {"Hub Tabanan": 18, "Hub Gianyar": 16, ...},
    "Hub Tabanan": {"Depot Denpasar": 18, "Zona Mengwi": 8, ...},
    ...
}
```

2.1.3 Jenis Node Dalam Sistem

Dalam sistem ini, simpul diklasifikasikan menjadi tiga tipe sesuai perannya dalam jaringan distribusi:

Tipe Node	Keterangan	Contoh dalam Sistem
Depot (Gudang Pusat)	Titik utama distribusi barang	Depot Denpasar
Hub (Transit)	Titik konsolidasi antara depot dan zona akhir	Hub Tabanan, Hub Gianyar, Hub Badung, Hub Klungkung
Zone (Drop Point)	Titik pengiriman akhir ke konsumen	Zona Kuta, Zona Ubud, Zona Nusa Penida, dll.

2.2 Decision Support System (DSS)

Decision Support System (DSS) adalah sistem informasi berbasis komputer yang dirancang untuk mendukung aktivitas pengambilan keputusan dalam organisasi. DSS menggabungkan data, model analitik, dan antarmuka pengguna yang interaktif untuk membantu pengambil keputusan dalam menghadapi situasi yang terstruktur maupun semi-terstruktur.

2.2.1 Komponen DSS

Sebuah DSS umumnya terdiri dari tiga komponen utama:

- Manajemen Data (Data Management): Basis data yang menyimpan informasi relevan. Dalam sistem ini, data disimpan sebagai struktur graf statis dalam file `graph.py` yang mendefinisikan `GRAPH` (adjacency list) dan `NODE_TYPES`.
- Manajemen Model (Model Management): Algoritma dan model analitik yang mengolah data. Dalam sistem ini, algoritma Dijkstra dalam file `dijkstra.py` berfungsi sebagai model komputasi untuk mencari rute terpendek.
- Antarmuka Pengguna (User Interface): Komponen yang memungkinkan pengguna berinteraksi dengan sistem. Diimplementasikan menggunakan Streamlit dalam file `app.py`, menampilkan sidebar input, visualisasi graf, dan panel hasil rute.

2.2.2 Karakteristik DSS

- Interaktif: Pengguna dapat memilih titik asal dan tujuan secara dinamis melalui dropdown menu.
- Berbasis Model: Keputusan rute didasarkan pada output algoritma Dijkstra, bukan intuisi atau perhitungan manual.
- Visual: Hasil rekomendasi disajikan dalam bentuk visualisasi graf yang menyoroti rute terpilih dengan warna berbeda.

- Informatif: Sistem menyajikan informasi lengkap meliputi total jarak, rincian setiap segmen, dan node transit yang dilewati.

2.3 Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma shortest-path klasik yang ditemukan oleh ilmuwan komputer Belanda Edsger W. Dijkstra pada tahun 1956. Algoritma ini digunakan untuk menemukan jalur terpendek dari satu simpul sumber ke semua simpul lainnya dalam graf berbobot dengan bobot non-negatif.

2.3.1 Prinsip Kerja

Algoritma Dijkstra bekerja dengan prinsip greedy, yaitu pada setiap iterasi memilih simpul dengan jarak sementara terkecil dari sumber. Langkah-langkah algoritma:

1. Inisialisasi: Tetapkan jarak semua simpul sebagai tak hingga (infinity), kecuali simpul sumber yang bernilai 0.
2. Gunakan priority queue (min-heap) untuk memproses simpul dengan jarak terkecil terlebih dahulu.
3. Untuk setiap simpul yang diproses, perbarui jarak tetangga jika ditemukan jalur yang lebih pendek (relaxation).
4. Tandai simpul yang telah diproses sebagai visited agar tidak diproses ulang.
5. Ulangi hingga semua simpul telah diproses atau simpul tujuan ditemukan.
6. Rekonstruksi jalur menggunakan array predecessor (previous) dari tujuan ke sumber, lalu balik urutannya.

2.3.2 Kompleksitas Algoritma

Implementasi	Kompleksitas Waktu	Kompleksitas Ruang
Array biasa (linear scan)	$O(V^2)$	$O(V)$
Binary Heap (heapq Python)	$O((V + E) \log V)$	$O(V + E)$
Fibonacci Heap	$O(E + V \log V)$	$O(V + E)$

Proyek ini menggunakan modul heapq Python yang mengimplementasikan binary min-heap, sehingga kompleksitas waktu mencapai $O((V + E) \log V)$. Dengan $V = 13$ node dan $E = 16$ edge, algoritma berjalan sangat cepat dalam hitungan milidetik.

2.3.3 Pseudo Code Dijkstra

```
function dijkstra(graph, start, end):  
    dist[all nodes] = infinity  
    dist[start] = 0  
    prev[all nodes] = null  
    pq = MinHeap()  
    pq.push((0, start))  
    visited = empty set  
  
    while pq is not empty:  
        (d, u) = pq.pop_min()  
        if u in visited: continue  
        visited.add(u)  
        if u == end: break  
        for each neighbor v of u with weight w:  
            if dist[u] + w < dist[v]:  
                dist[v] = dist[u] + w  
                prev[v] = u  
                pq.push((dist[v], v))  
  
    return reconstruct_path(prev, start, end), dist[end]
```

2.3.4 Library dan Teknologi yang Digunakan

Library	Versi	Fungsi
Python	3.9 / 3.12	Bahasa pemrograman utama
Streamlit	Latest	Framework antarmuka web interaktif
NetworkX	Latest	Pembuatan dan manipulasi graf
Matplotlib	Latest	Visualisasi grafis graf dan rute
heapq	Built-in	Implementasi priority queue untuk Dijkstra

BAB III

ANALISIS DAN PERANCANGAN

3.1 Analisis Masalah

Permasalahan utama yang diselesaikan dalam proyek ini adalah: Diberikan jaringan distribusi logistik yang terdiri dari depot, hub transit, dan zona pengiriman, tentukan jalur terpendek (dalam satuan kilometer) dari titik asal ke titik tujuan yang dipilih pengguna.

Dari analisis kebutuhan, sistem harus memenuhi persyaratan berikut:

3.1.1 Persyaratan Fungsional

- Sistem dapat menerima input berupa pilihan node asal dan node tujuan dari pengguna.
- Sistem menghitung rute terpendek menggunakan algoritma Dijkstra.
- Sistem menampilkan rute lengkap (daftar node yang dilalui) beserta total jarak dalam km.
- Sistem menampilkan rincian setiap segmen (dari node mana ke node mana, dengan jarak berapa km).
- Sistem memvisualisasikan graf jaringan dengan node dan edge, menyoroti rute terpendek.
- Sistem menangani kondisi error: asal sama dengan tujuan, atau tidak ada jalur yang menghubungkan.

3.1.2 Persyaratan Non-Fungsional

- Performa: Waktu komputasi rute harus kurang dari 1 detik untuk jaringan skala proyek ini.
- Usability: Antarmuka harus intuitif, dapat digunakan tanpa pelatihan teknis khusus.
- Portabilitas: Sistem harus dapat dijalankan di sistem operasi manapun yang mendukung Python 3.x.
- Reliabilitas: Sistem menggunakan session_state Streamlit agar hasil tidak hilang saat komponen UI diperbarui.

3.2 Desain Graf

Graf dirancang untuk merepresentasikan jaringan distribusi logistik di Pulau Bali dengan karakteristik sebagai berikut:

3.2.1 Topologi Jaringan

- Tipe: Undirected weighted graph (graf tak berarah berbobot).
- Jumlah node (V): 13 node, terdiri dari 1 depot, 4 hub, dan 8 zona.
- Jumlah edge (E): 16 edge unik (sisi bersifat dua arah, jadi ada 32 koneksi directional).
- Bobot edge: Jarak dalam satuan kilometer, bernilai antara 4 km (Hub Klungkung – Zona Semarapura) hingga 35 km (Hub Tabanan – Zona Semarapura).

3.2.2 Daftar Edge dan Bobot

Node A	Node B	Jarak (km)
Depot Denpasar	Hub Tabanan	18
Depot Denpasar	Hub Gianyar	16
Depot Denpasar	Hub Badung	12
Depot Denpasar	Hub Klungkung	28
Hub Tabanan	Hub Gianyar	22
Hub Tabanan	Zona Mengwi	8
Hub Tabanan	Zona Kediri	6
Hub Tabanan	Zona Semarapura	35
Hub Gianyar	Zona Ubud	7
Hub Gianyar	Zona Sukawati	5
Hub Gianyar	Zona Jimbaran	27
Hub Badung	Hub Klungkung	32
Hub Badung	Zona Kuta	6
Hub Badung	Zona Jimbaran	9
Hub Klungkung	Zona Semarapura	4
Hub Klungkung	Zona Nusa Penida	18

3.3 Flowchart Sistem

Berikut adalah alur kerja utama sistem DSS secara keseluruhan:

3.3.1 Flowchart Alur Utama Aplikasi

- MULAI: Pengguna membuka aplikasi Streamlit.
- Sistem menginisialisasi graf jaringan dari modul graph.py.
- Antarmuka sidebar menampilkan dropdown untuk memilih node asal dan tujuan.
- Pengguna memilih node asal dan node tujuan, lalu menekan tombol "Cari Rute Terpendek".
- Sistem memvalidasi input: apakah node asal sama dengan tujuan? Jika ya, tampilkan peringatan.
- Sistem memanggil fungsi dijkstra(start, end) dari modul dijkstra.py.
- Algoritma mengembalikan (path, distance) atau (None, infinity) jika tidak ada jalur.
- Jika ada jalur: simpan hasil ke session_state, render visualisasi graf dengan rute yang disorot, tampilkan panel info berisi total jarak dan step-by-step.
- SELESAI: Pengguna dapat melakukan pencarian baru dengan memilih node yang berbeda.

3.3.2 Flowchart Algoritma Dijkstra

- MULAI: Terima input start dan end.
- Inisialisasi: dist[semua] = infinity, dist[start] = 0, prev[semua] = None, visited = set kosong, priority queue = [(0, start)].
- Loop: Ambil (current_dist, current_node) dari priority queue.
- Cek: Apakah current_node sudah dalam visited? Jika ya, lanjut iterasi berikutnya.
- Tambah current_node ke visited.
- Cek: Apakah current_node == end? Jika ya, keluar dari loop.
- Iterasi tetangga: Untuk setiap (neighbor, weight) dari current_node, hitung new_dist = current_dist + weight. Jika new_dist < dist[neighbor], perbarui dist[neighbor] dan prev[neighbor], masukkan ke priority queue.
- Rekonstruksi: Telusuri prev dari end ke start, balik array path.
- Return: (path, dist[end]) atau (None, infinity).

3.4 Use Case

Diagram use case menggambarkan interaksi antara aktor (pengguna sistem) dan fungsionalitas yang tersedia:

3.4.1 Aktor

- Operator Logistik: Pengguna utama sistem yang bertugas merencanakan dan mengelola rute pengiriman.

3.4.2 Use Case Utama

No	Use Case	Deskripsi
UC01	Memilih Node Asal	Operator memilih titik keberangkatan dari dropdown yang berisi semua node dalam jaringan.
UC02	Memilih Node Tujuan	Operator memilih titik tujuan pengiriman dari dropdown yang sama.
UC03	Mencari Rute Terpendek	Sistem menghitung dan menampilkan rute terpendek setelah tombol diklik.
UC04	Melihat Visualisasi Graf	Operator melihat visualisasi grafis jaringan dengan rute yang disorot.
UC05	Melihat Detail Rute	Operator membaca informasi step-by-step perjalanan beserta jarak segmen.
UC06	Melihat Node Transit	Operator mengetahui hub-hub mana yang disinggahi dalam perjalanan.

3.5 Struktur Node dan Edge

3.5.1 Struktur Data Node

Setiap node dalam sistem memiliki dua atribut:

- Nama (string): Identifikasi unik node, contohnya "Depot Denpasar", "Hub Tabanan", "Zona Kuta".
- Tipe (string): Salah satu dari tiga nilai: "depot", "hub", atau "zone". Tipe menentukan warna representasi visual node.

Implementasi dalam kode:

```
NODE_TYPES = {  
    "Depot Denpasar": "depot",  
    "Hub Tabanan": "hub",  
    "Zona Kuta": "zone",  
    ...  
}
```

3.5.2 Struktur Data Edge

Setiap edge merepresentasikan koneksi dua arah antara dua node dengan bobot jarak (km). Edge disimpan dalam adjacency list yang diimplementasikan sebagai nested dictionary. Karena graf bersifat undirected, setiap koneksi (A, B, w) didefinisikan dua kali: A->B dan B->A dengan bobot yang sama.

3.5.3 Struktur Output Dijkstra

Fungsi dijkstra() mengembalikan dua nilai:

- path (list of string): Urutan nama node dari start ke end, contohnya ["Depot Denpasar", "Hub Badung", "Zona Kuta"].
- distance (int/float): Total jarak dalam km, atau float('inf') jika tidak ada jalur.

BAB IV

IMPLEMENTASI

4.1 Struktur Proyek

Proyek diorganisasikan dalam struktur direktori berikut:

```
TugasUASStrukturData/
|-- app.py          # Entry point: antarmuka Streamlit
|-- requirements.txt # Daftar dependensi Python
|-- core/
    |-- __init__.py  # Package initializer
    |-- graph.py     # Definisi graf: NODE_TYPES, GRAPH, fungsi helper
    |-- dijkstra.py  # Implementasi algoritma Dijkstra
    |-- visualizer.py # Fungsi visualisasi graf dengan NetworkX + Matplotlib
```

4.2 Penjelasan Kode

4.2.1 Modul graph.py – Struktur Data Graf

File ini merupakan inti dari representasi data sistem. Terdapat tiga komponen utama:

Pertama, dictionary `NODE_TYPES` mendefinisikan tipe setiap node. Ini digunakan untuk menentukan warna node dalam visualisasi dan badge dalam antarmuka.

Kedua, dictionary `GRAPH` mengimplementasikan adjacency list berbobot. Graf mencakup 13 node dengan 16 edge unik, seluruhnya merepresentasikan jaringan distribusi nyata di Bali.

Ketiga, fungsi-fungsi helper: `get_neighbors(node)` mengembalikan tetangga satu node, `get_all_nodes()` mengembalikan daftar semua node untuk dropdown UI, dan `get_all_edges()` mengembalikan semua edge unik (menghindari duplikasi undirected).

4.2.2 Modul dijkstra.py – Algoritma Dijkstra

File ini mengimplementasikan algoritma Dijkstra menggunakan `heapq` Python. Fungsi utama `dijkstra(start, end)` bekerja sebagai berikut:

```
def dijkstra(start, end):
    distances = {node: float('inf') for node in GRAPH}
    distances[start] = 0
    previous = {node: None for node in GRAPH}
    queue = [(0, start)]
```

```

visited = set()

while queue:
    current_dist, current_node = heapq.heappop(queue)
    if current_node in visited: continue
    visited.add(current_node)
    if current_node == end: break
    for neighbor, weight in GRAPH[current_node].items():
        new_dist = current_dist + weight
        if new_dist < distances[neighbor]:
            distances[neighbor] = new_dist
            previous[neighbor] = current_node
            heapq.heappush(queue, (new_dist, neighbor))
# Rekonstruksi jalur
path = []
current = end
while current is not None:
    path.append(current)
    current = previous[current]
path.reverse()
if path[0] != start: return None, float('inf')
return path, distances[end]

```

Fungsi tambahan `get_step_by_step(path)` mengubah daftar node menjadi daftar segmen (dari-ke) yang digunakan UI untuk menampilkan rincian perjalanan.

4.2.3 Modul `visualizer.py` – Visualisasi Graf

File ini menggunakan NetworkX untuk membangun objek graf dan Matplotlib untuk merendernya. Tiga fungsi utama:

- `build_nx_graph()`: Membuat objek NetworkX Graph dari `GRAPH` dan `NODE_TYPES`. Setiap node diberi atribut `node_type`, dan setiap edge diberi atribut `weight`.
- `get_layout(G)`: Mengembalikan posisi manual setiap node dalam koordinat 2D yang merepresentasikan posisi geografis relatif di Pulau Bali.
- `draw_graph(path, title)`: Fungsi utama yang menggambar graf. Jika parameter `path` diisi, edge-edge dalam jalur tersebut akan digambar dengan warna oranye tebal (`#F59E0B`,

lebar 4.5pt), sementara edge lain berwarna abu-abu (#CBD5E1). Node dalam jalur juga diberi highlight lebih besar. Label node dan bobot edge ditampilkan untuk kemudahan baca.

4.2.4 Modul `app.py` – Antarmuka Streamlit

File utama yang mengintegrasikan semua komponen. Terdiri dari beberapa bagian:

- Konfigurasi halaman: `st.set_page_config()` mengatur judul, ikon, dan layout lebar.
- CSS kustom: Mendefinisikan tampilan visual dengan palet warna navy-oranye yang konsisten, termasuk badge tipe node berwarna merah (depot), biru (hub), dan hijau (zone).
- Sidebar: Memuat dua dropdown (node asal dan tujuan) dan tombol aksi. Menampilkan badge tipe node dan legenda warna.
- State management: Menggunakan `st.session_state` untuk menyimpan hasil pencarian agar tidak hilang saat widget lain berinteraksi.
- Area utama: Layout dua kolom – kolom kiri (lebar 2.2x) menampilkan visualisasi graf, kolom kanan (lebar 1x) menampilkan kartu hasil berisi total jarak, jalur lengkap, rincian segmen, dan node transit.

4.3 Tampilan Sistem

4.3.1 Halaman Utama (Sebelum Pencarian)

Saat aplikasi pertama kali dibuka, halaman menampilkan:

- Header berjudul "DSS Rute Pengiriman — Distribusi Bali" dengan subjudul tentang algoritma Dijkstra.
- Sidebar berisi dropdown kosong (node asal dan tujuan belum dipilih).
- Visualisasi graf penuh jaringan distribusi Bali tanpa sorotan rute, menampilkan seluruh 13 node dengan warna sesuai tipe dan semua 16 edge dengan label jarak.
- Panel kanan menampilkan instruksi penggunaan dan daftar semua node dalam jaringan.
-

4.3.2 Halaman Hasil Pencarian

Setelah pengguna menekan tombol "Cari Rute Terpendek", tampilan berubah menjadi:

- Visualisasi graf yang sama namun dengan rute terpendek yang disorot menggunakan garis tebal berwarna oranye (#F59E0B). Node-node dalam jalur juga ditampilkan lebih besar.
- Kartu hasil di panel kanan menampilkan: total jarak dengan font besar berwarna biru, informasi asal dan tujuan beserta badge tipe node, jalur lengkap berupa urutan node yang dihubungkan panah, rincian setiap segmen beserta jaraknya dalam km, dan daftar node transit yang dilewati.

4.3.3 Penanganan Error

- Jika asal = tujuan: Muncul pesan warning kuning "Titik asal dan tujuan tidak boleh sama."

- Jika tidak ada jalur: Muncul pesan error merah "Tidak ditemukan rute yang menghubungkan kedua titik ini."

4.4 Cara Menjalankan Sistem

Untuk menjalankan sistem, ikuti langkah-langkah berikut:

5. Pastikan Python 3.9 atau lebih baru telah terinstal.
6. Install dependensi: `pip install -r requirements.txt`
7. Jalankan aplikasi: `streamlit run app.py`
8. Buka browser dan akses alamat yang ditampilkan (biasanya `http://localhost:8501`).

BAB V

SKENARIO

5.1 Skenario Pengujian

Pengujian dilakukan dengan menjalankan sistem dan memverifikasi output untuk berbagai skenario. Setiap skenario diverifikasi secara manual dengan menghitung jarak berdasarkan adjacency list.

5.1.1 Skenario 1: Rute Depot ke Zona Terdekat

Input: Asal = Depot Denpasar, Tujuan = Zona Kuta

Jalur yang diharapkan: Depot Denpasar -> Hub Badung -> Zona Kuta

Jarak yang diharapkan: $12 + 6 = 18$ km

Hasil sistem: Depot Denpasar -> Hub Badung -> Zona Kuta, 18 km [LULUS]

5.1.2 Skenario 2: Rute Melewati Multiple Hub

Input: Asal = Depot Denpasar, Tujuan = Zona Nusa Penida

Jalur yang diharapkan: Depot Denpasar -> Hub Klungkung -> Zona Nusa Penida

Jarak: $28 + 18 = 46$ km

Alternatif via Hub Badung: $12 + 32 + 18 = 62$ km (lebih panjang)

Hasil sistem: Depot Denpasar -> Hub Klungkung -> Zona Nusa Penida, 46 km [LULUS]

5.1.3 Skenario 3: Rute Antar Zona (Bukan Depot ke Zona)

Input: Asal = Zona Kuta, Tujuan = Zona Ubud

Analisis jalur: Zona Kuta -> Hub Badung (6) -> Depot Denpasar (12) -> Hub Gianyar (16) -> Zona Ubud (7) = 41 km

Alternatif: Zona Kuta -> Hub Badung (6) -> Hub Klungkung (32) -> Depot Denpasar (28) -> Hub Gianyar (16) -> Zona Ubud (7) = 89 km

Hasil sistem: Zona Kuta -> Hub Badung -> Depot Denpasar -> Hub Gianyar -> Zona Ubud, 41 km [LULUS]

5.1.4 Skenario 4: Rute Zona ke Zona yang Berdekatan Secara Hub

Input: Asal = Zona Mengwi, Tujuan = Zona Kediri

Analisis: Zona Mengwi -> Hub Tabanan (8) -> Zona Kediri (6) = 14 km

Hasil sistem: Zona Mengwi -> Hub Tabanan -> Zona Kediri, 14 km [LULUS]

5.1.5 Skenario 5: Input Error – Asal Sama dengan Tujuan

Input: Asal = Depot Denpasar, Tujuan = Depot Denpasar

Hasil yang diharapkan: Pesan peringatan, tidak ada perhitungan.

Hasil sistem: Menampilkan st.warning("Titik asal dan tujuan tidak boleh sama.") [LULUS]

5.2 Analisis Hasil

No	Asal	Tujuan	Jarak	Jumlah Hop	Status
1	Depot Denpasar	Zona Kuta	18 km	2 hop	LULUS
2	Depot Denpasar	Zona Nusa Penida	46 km	2 hop	LULUS
3	Depot Denpasar	Zona Semarapura	32 km	2 hop	LULUS
4	Zona Kuta	Zona Ubud	41 km	4 hop	LULUS
5	Zona Mengwi	Zona Kediri	14 km	2 hop	LULUS
6	Zona Nusa Penida	Zona Kuta	58 km	3 hop	LULUS
7	Asal = Tujuan	—	—	—	Warning

Seluruh skenario pengujian menghasilkan output yang sesuai dengan perhitungan manual. Algoritma Dijkstra terbukti menghasilkan jalur yang benar-benar optimal (bukan hanya mendekati optimal) untuk semua kombinasi node yang diuji.

5.3 Kompleksitas Algoritma

5.3.1 Analisis Teoritis

Implementasi menggunakan heapq (binary min-heap) memberikan kompleksitas:

- Waktu: $O((V + E) \log V)$ di mana V = jumlah node, E = jumlah edge.
- Ruang: $O(V + E)$ untuk menyimpan distances, previous, priority queue, dan visited set.

5.3.2 Analisis Empiris untuk Jaringan Ini

Dengan parameter konkret:

- $V = 13$ node, $E = 32$ koneksi directional (16 edge x 2 karena undirected).
- $(V + E) \log V = (13 + 32) \times \log(13) = 45 \times 3.7 \approx 166$ operasi.

- Dalam pengujian aktual, seluruh pencarian selesai dalam waktu di bawah 5 milidetik.

5.3.3 Perbandingan Pendekatan

Pendekatan	Kompleksitas Waktu	Optimal?	Cocok untuk Proyek Ini?
Brute Force (semua jalur)	$O(V!)$	Ya	Tidak
BFS (tanpa bobot)	$O(V + E)$	Tidak (bobot diabaikan)	Tidak
Dijkstra + Array	$O(V^2)$	Ya	Bisa, tapi tidak efisien
Dijkstra + Min-Heap	$O((V+E) \log V)$	Ya	Ya (digunakan)
A* (heuristic)	$O(E)$	Ya (heuristic)	Bisa, namun overkill

Dijkstra dengan min-heap adalah pilihan optimal untuk skala proyek ini karena menjamin hasil terpendek, kompleksitas waktu logaritmik, dan kemudahan implementasi menggunakan library bawaan Python (heapq).

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

- Jaringan distribusi logistik di Pulau Bali berhasil dimodelkan sebagai graf berbobot tak berarah menggunakan struktur adjacency list berbasis Python dictionary. Model ini mencakup 13 node (1 depot, 4 hub, 8 zona) dan 16 edge dengan bobot jarak dalam satuan kilometer.
- Algoritma Dijkstra berhasil diimplementasikan menggunakan heapq Python sebagai priority queue, menghasilkan rute terpendek yang optimal untuk semua kombinasi pasangan node yang diuji. Seluruh 7 skenario pengujian menunjukkan hasil yang benar sesuai perhitungan manual.
- Antarmuka DSS berbasis Streamlit berhasil dikembangkan dengan tampilan yang intuitif, responsif, dan informatif. Pengguna dapat dengan mudah memilih node asal-tujuan, mendapatkan rekomendasi rute instan, dan memahami hasil melalui visualisasi grafis serta rincian step-by-step.
- Integrasi visualisasi graf menggunakan NetworkX dan Matplotlib menghasilkan representasi grafis yang efektif, di mana rute terpendek disorot dengan jelas menggunakan warna dan ketebalan garis yang berbeda.
- Sistem menunjukkan performa eksekusi yang sangat baik, dengan waktu komputasi di bawah 5 milidetik untuk jaringan skala proyek, jauh memenuhi persyaratan performa yang ditetapkan.
- Penerapan session_state Streamlit memastikan stabilitas UI sehingga hasil pencarian tidak hilang saat komponen antarmuka lain diperbarui, memberikan pengalaman pengguna yang mulus.

6.2 Saran Pengembangan

Untuk pengembangan lebih lanjut, beberapa aspek berikut dapat dipertimbangkan:

6.2.1 Peningkatan Data dan Model

- Integrasi Data Real-time: Mengintegrasikan API peta (seperti Google Maps API atau OpenStreetMap) untuk mendapatkan data jarak aktual dan memperbarui bobot edge secara dinamis berdasarkan kondisi lalu lintas.
- Faktor Multi-kriteria: Memperluas model DSS untuk mempertimbangkan bobot multi-dimensi seperti waktu tempuh, biaya bahan bakar, kapasitas kendaraan, dan kondisi jalan secara bersamaan.
- Penambahan Node: Memperluas cakupan jaringan ke seluruh kabupaten/kota di Bali, termasuk Karangasem, Buleleng, Jembrana, dan Bangli.

6.2.2 Peningkatan Algoritma

- Implementasi A* Algorithm: Mengimplementasikan algoritma A* dengan heuristic berbasis jarak Euclidean atau koordinat GPS untuk potensi percepatan komputasi pada jaringan yang lebih besar.
- K-Shortest Paths: Menambahkan fitur pencarian k-jalur terpendek (misal: 3 alternatif rute) agar operator dapat memilih berdasarkan preferensi tambahan.
- Vehicle Routing Problem (VRP): Mengembangkan sistem ke arah penyelesaian VRP, di mana satu depot harus melayani banyak tujuan sekaligus dengan armada kendaraan terbatas.

6.2.3 Peningkatan Sistem

- Persistensi Data: Mengimplementasikan database (SQLite atau PostgreSQL) untuk menyimpan histori pencarian rute, memungkinkan analisis pola distribusi.
- Autentikasi Pengguna: Menambahkan sistem login agar berbagai level pengguna (operator, manajer, admin) memiliki akses dan tampilan yang sesuai perannya.
- Export Laporan: Menambahkan fitur ekspor hasil pencarian ke format PDF atau Excel untuk keperluan pelaporan dan dokumentasi operasional.
- Deployment Cloud: Mendeploy aplikasi ke platform cloud seperti Streamlit Community Cloud, Heroku, atau AWS untuk aksesibilitas yang lebih luas tanpa instalasi lokal.

6.2.4 Validasi dan Pengujian

- Pengujian dengan Data Dunia Nyata: Memvalidasi akurasi sistem dengan membandingkan rekomendasi rute terhadap data perjalanan aktual dari armada logistik yang beroperasi di Bali.

Stress Testing: Menguji performa sistem pada jaringan yang jauh lebih besar (ratusan hingga ribuan node) untuk memastikan skalabilitas arsitektur.